

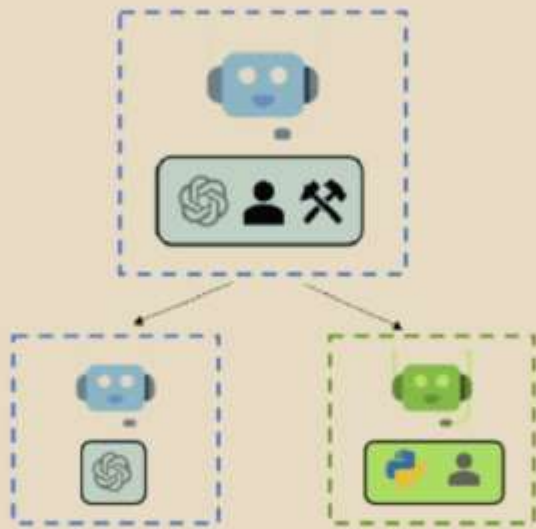
Advanced Natural Language Processing

Exploring Multi-Agent Systems with AutoGen

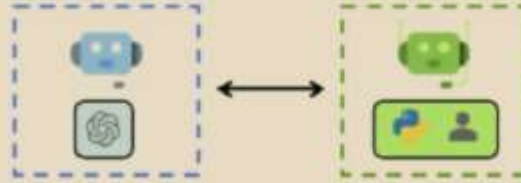
Sher Muhammad Daudpota
Sukkur IBA University

AutoGen uses conversable agents, which communicate through conversations.

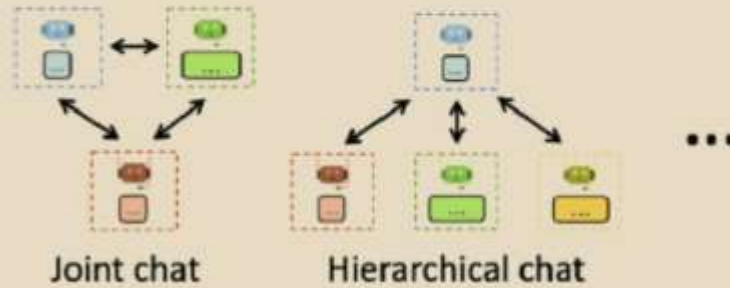
Conversable agent



Agent Customization



Multi-Agent Conversations



Flexible Conversation Patterns

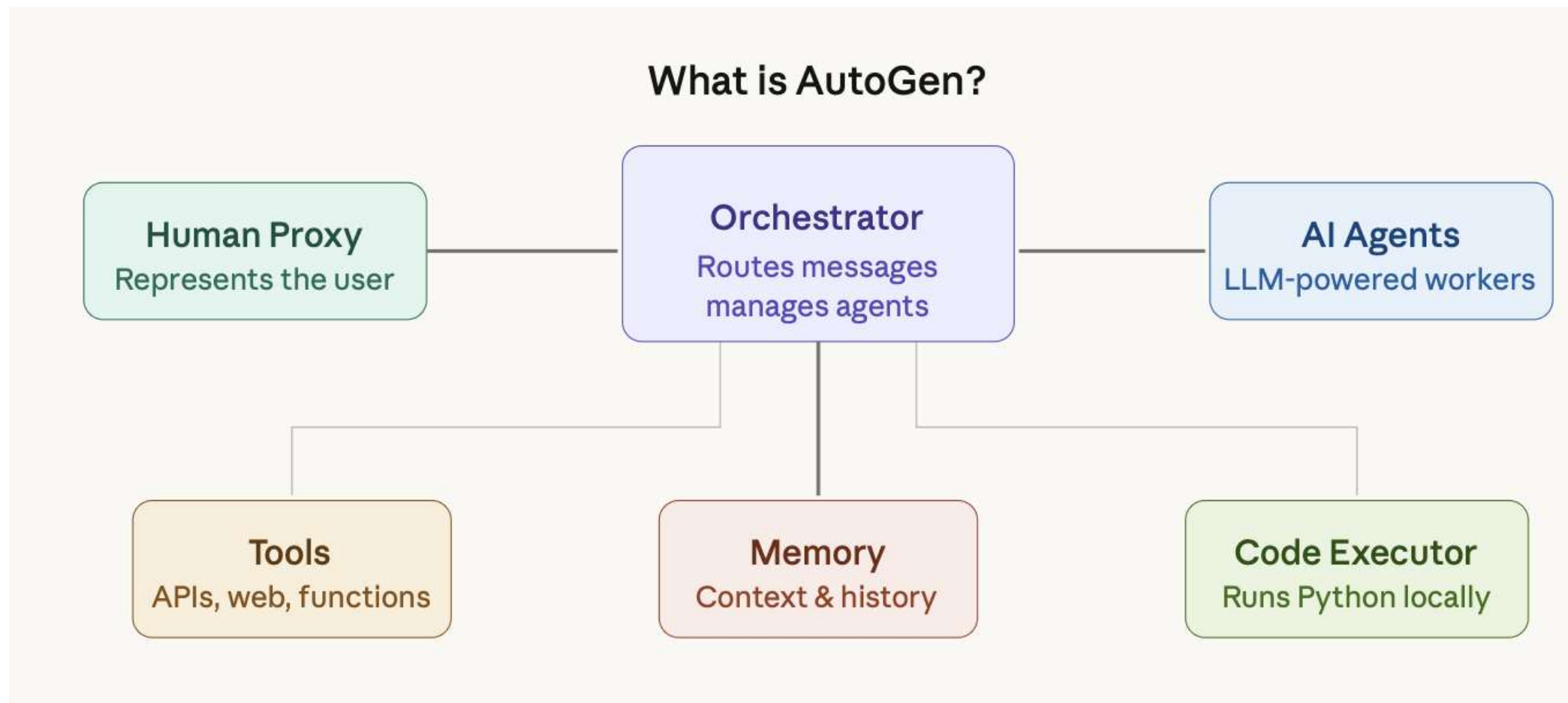
Figure 4.1 How AutoGen agents communicate through conversations
(Source: AutoGen)

Multi-
Agent
with
AutoGen



Figure 4.2 The user proxy agent and assistant agent communication (Source: AutoGen)

So what's autogen?



So what's autogen?

AutoGen is a framework by Microsoft that lets you build systems where multiple AI agents collaborate to solve complex tasks. Instead of one LLM doing everything, you design a *team* of specialized agents that can talk to each other, write code, browse the web, and call tools.

Key concepts:

- Agents = autonomous units with a role, a system prompt, and access to tools
- Conversations = structured message passing between agents
- Human-in-the-loop = optionally approve/intervene at any step

Installation & setup

pip install
pyautogen

Set API Key
OpenAI / local

Create Agents
Define roles & config

Start Chat
initiate_chat()

Code:

```
autogen_complete_totutorial.ipyn
```

```
b
```

```
colab
```

Now, we will work within
pycharm

Two-Way Chat with AutoGen

● Conversational AI Agents

Two agents exchange messages in a structured loop — one acts as the initiator, the other as a responder

● Automatic Reply Mechanism

AutoGen's `initiate_chat()` drives the conversation, handling turn-taking and message passing automatically

● Customizable Termination

Define stopping rules via `is_termination_msg` or `max_turns` to control when the dialogue ends

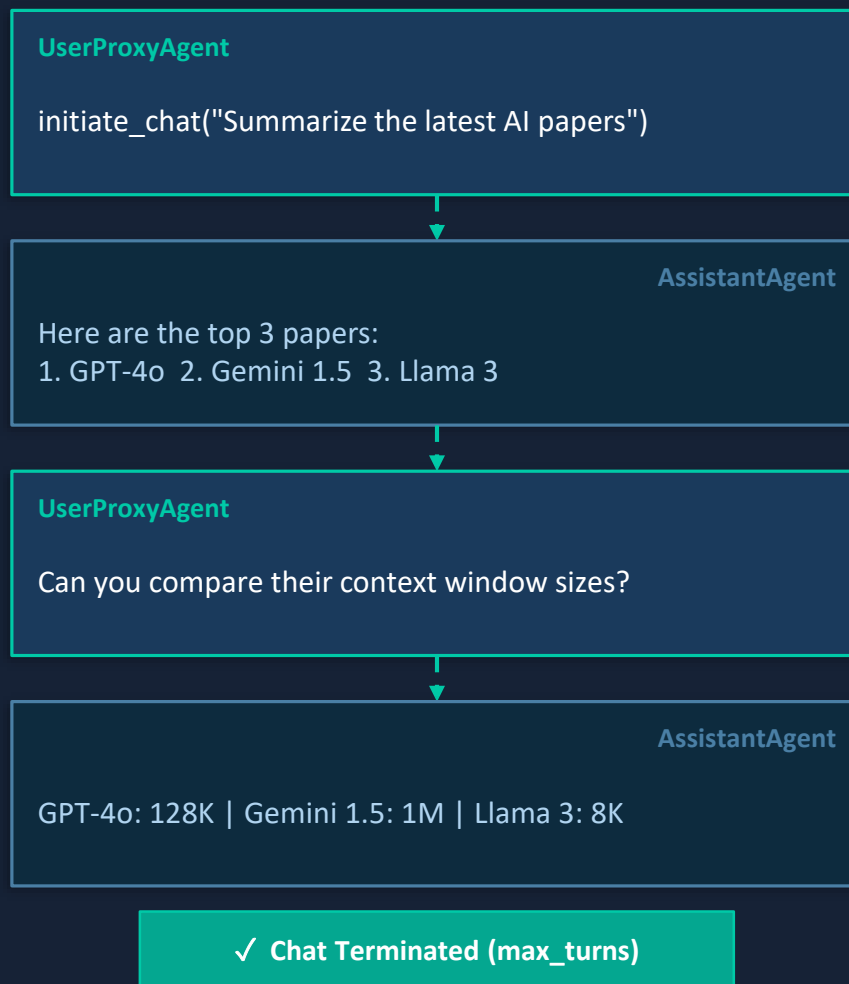
● Human-in-the-Loop Support

Optionally inject human feedback at any turn with `human_input_mode` set to `ALWAYS`, `NEVER`, or `TERMINATE`

● Tool & Code Execution

Agents can call tools, run code, and share results within the same chat session seamlessly

Chat Flow



Two-Way Chat



- **Create Conda Env**
- **Installing pyautogen**
- **Creating configuration to connect to LLM**
- **Create the Agents**
- **Start the Chat!**

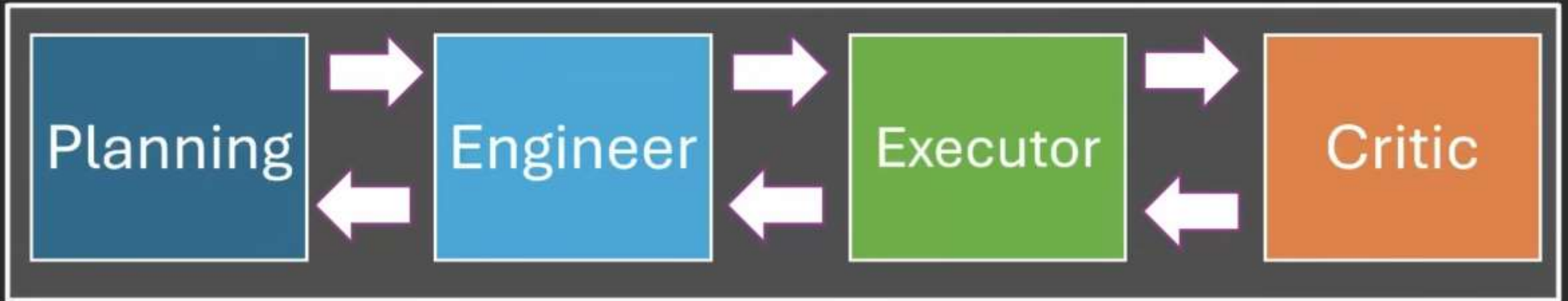
Human Interaction



- We can change the configuration for the **UserAgent**
- `human_input_mode="ALWAYS"`

Group Chat

User



Sequential Chat

UserAgent



Assistant
Poem1

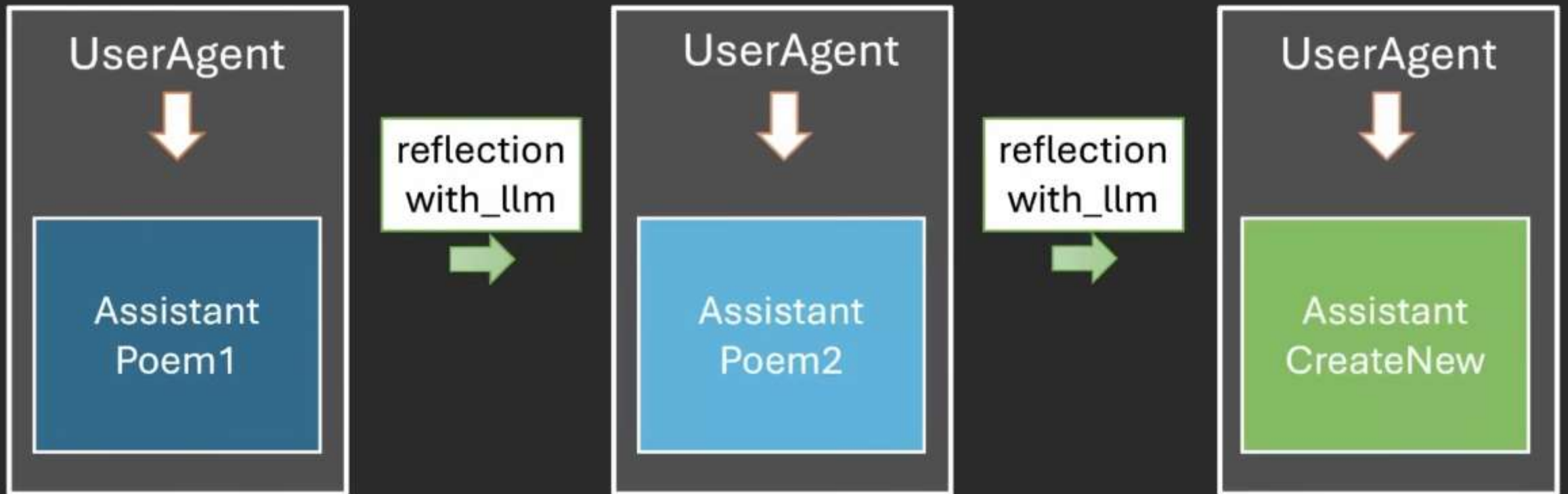


Assistant
Poem2

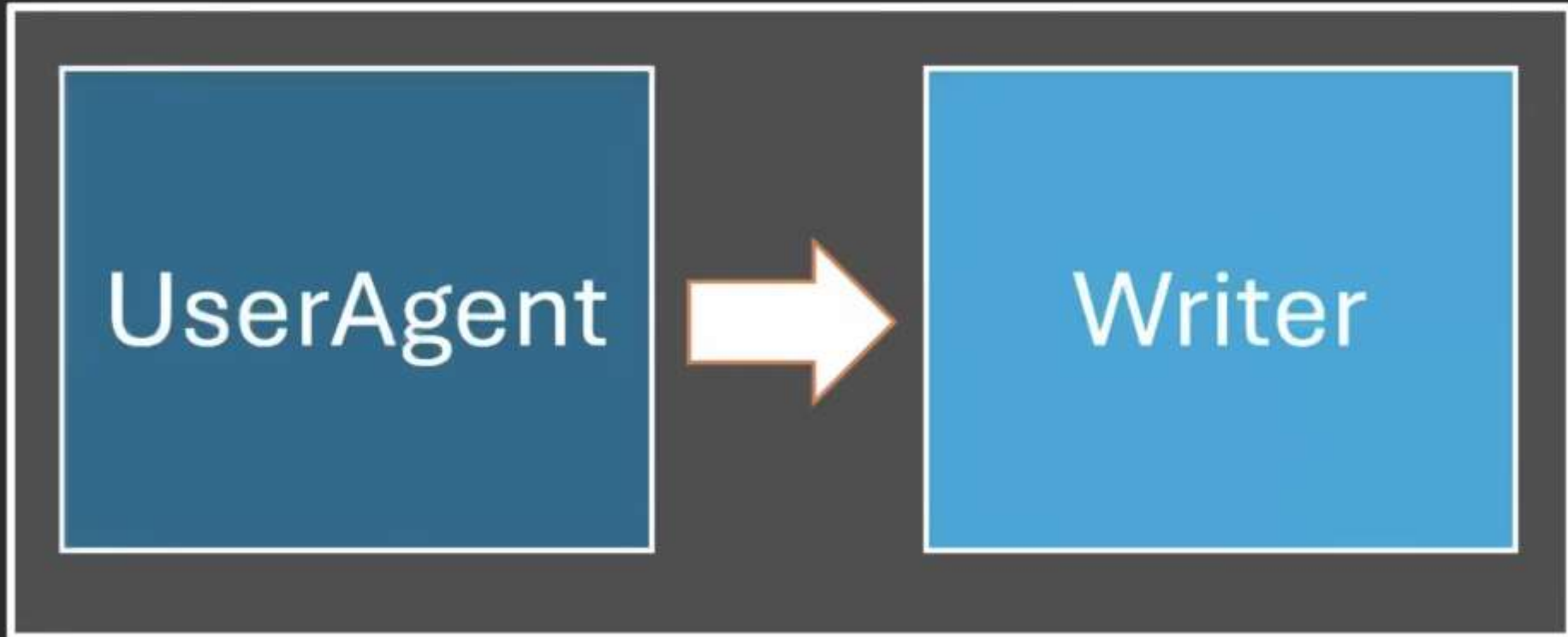


Assistant
CreateNew

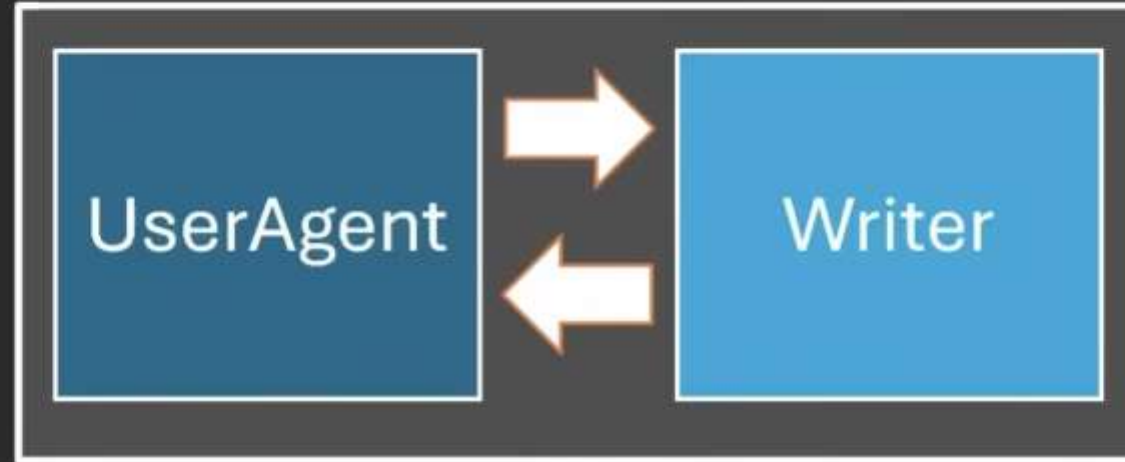
Sequential Chat



Nested Chat



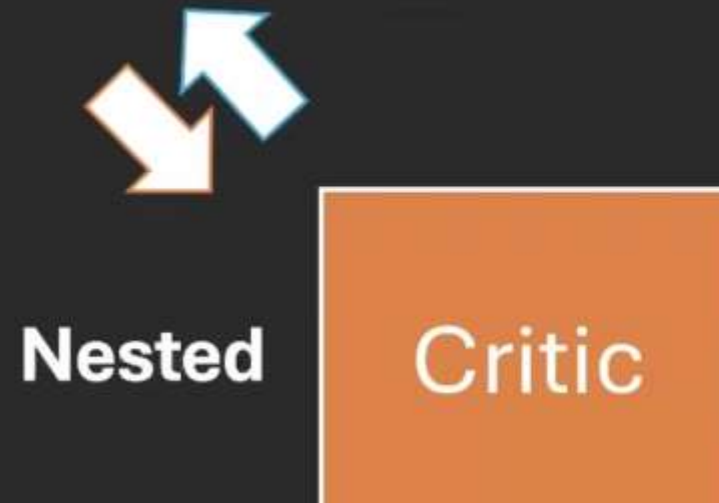
Nested Chat



When **writer** is done, we want the critic to check the work

Repeatable – this is 2 'rounds'

1. **User -> Writer**
2. **Writer -> User**
3. **User -> Critic**
4. **Critic -> User**



Performance Analysis

- How long was the chat?
- How many tokens did we use?
- What was the cost?
- Storing the response?

Start
Log



Stop
Log



Researcher



Writer



Critic

How it looks in Code

```
# Start logging
logging_session_id = autogen.runtime_logging.start(config={"dbname": "logs.db"})
print("Logging session ID: " + str(logging_session_id))
```

```
{
    ....
    logic
    ....
}
```

```
autogen.runtime_logging.stop()
```

Retrieving Data

id	invocation_id	client_id	wrapper_id	session_id	request	response	is_cached	cost	start_time	end_time
Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1efb9433-af66-4a2a-95df-001157109161	6350296976	6333967312	1180d5c3-39cd-476c-8708-4f950c72b63e	{"messages": [{"content": "You are a helpful AI assistant..."	{...	0	0.0	2024-02-26 17:48:11.610533	2024-02-26 17:48:12.659182
2	5eed498d-cf9b-44e0-93a2-c785efcd7f18	6408004368	6406989136	8ddc710b-6d7d-465a-bd6d-485edc7d5fc8	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.0	2024-02-26 17:48:59.850395	2024-02-26 17:48:59.851077
3	621ef54d-94c8-4461-82e7-5dcd6b2fc2c2	6382185168	6370638992	833d3470-3f2c-4fb8-b662-c2012f08735c	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.0	2024-02-26 17:49:16.180570	2024-02-26 17:49:16.182042
4	8aabf0cb-2c1a-49b5-995f-d2c75737798c	6410017744	4522790160	a0c3f90a-36b7-4e9f-af0e-2680a801121d	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.0	2024-02-26 17:49:37.264896	2024-02-26 17:49:37.265786
5	da3326a4-012c-4e11-b31b-26debc938401	6414475408	6401932880	402f8f95-b451-4e9f-b7cb-885438196c0a	{"messages": [{"content": "You are a helpful AI assistant..."	{...	0	0.02766	2024-02-26 17:54:43.353215	2024-02-26 17:54:56.794278
6	edc13700-448f-41bb-81b7-c07a3ac374a2	6414475408	6401932880	402f8f95-b451-4e9f-b7cb-885438196c0a	{"messages": [{"content": "You are a helpful AI assistant..."	{...	0	0.02487	2024-02-26 17:54:55.802681	2024-02-26 17:54:58.597052
7	bc018c57-de89-44e0-99a8-8b641b6c8914	6349515408	6341097936	a2c69493-9ff9-43cf-bd84-104d57d2c933	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.02766	2024-02-26 18:00:39.069701	2024-02-26 18:00:39.071419
8	d807bbd8-2aac-4b3f-8163-f4ff79a83601	6349515408	6341097936	a2c69493-9ff9-43cf-bd84-104d57d2c933	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.02487	2024-02-26 18:00:39.076102	2024-02-26 18:00:39.076282
9	a8c4c7ea-8895-45fe-b0ff-cf45999a1ae9	6386412752	6369892368	a405bd41-3d15-4686-a133-7622e7e981de	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.02766	2024-02-26 18:00:46.997054	2024-02-26 18:00:46.997672
10	db464dc0-ebeb-469d-80e6-5799442c43cb	6386412752	6369892368	a405bd41-3d15-4686-a133-7622e7e981de	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.02487	2024-02-26 18:00:47.001014	2024-02-26 18:00:47.001122
11	3ae86093-79ef-45c7-b7ae-a6d340510498	6376728016	6366182608	2f68629b-7afa-4ff4-86c3-6f293aa6d54b	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.02766	2024-02-26 18:02:15.158942	2024-02-26 18:02:15.159985
12	cfb84a4f-d314-45fe-8e20-65df7e87bd80	6376728016	6366182608	2f68629b-7afa-4ff4-86c3-6f293aa6d54b	{"messages": [{"content": "You are a helpful AI assistant..."	{...	1	0.02487	2024-02-26 18:02:15.162792	2024-02-26 18:02:15.162921
13	2911cc94-e4cd-4ed2-af03-711462526a11	6340159312	6341196880	208a677d-538e-4686-a113-...	{"messages": [{"content": "You are a helpful AI assistant..."	{...	0	0.03414	2024-02-26 18:02:28.373444	2024-02-26 18:02:40.216550
14	f8201a6a-1ca3-4b5e-895c-c6b9ffca2efa	6340159312	6341196880	208a677d-538e-4686-a113-...	{"messages": [{"content": "You are a helpful AI assistant..."	{...	0	0.02559	2024-02-26 18:02:40.221861	2024-02-26 18:02:41.426781

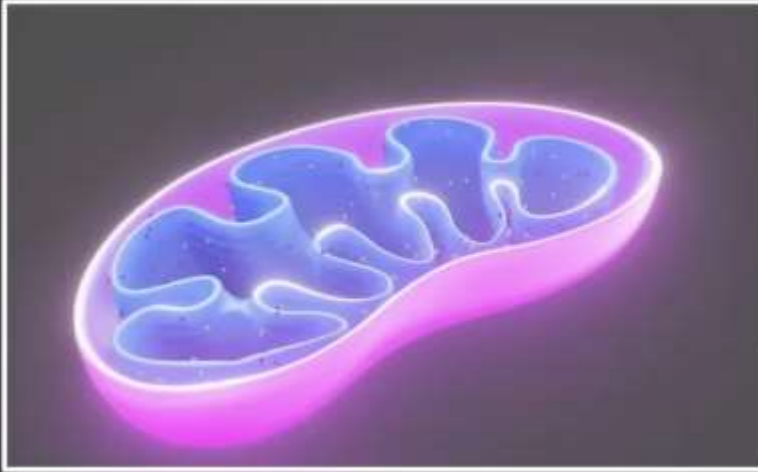


SQLiteBrowser

Vision Agent

- image-2-text
- describe an image
- GPT-4-Vision-Preview

Vision Model



Output:

The image features a stylized, three-dimensional representation of a **mitochondrion**, an organelle often found in eukaryotic cells. It is depicted with a vibrant and somewhat neon color scheme, giving it an almost futuristic or sci-fi appearance.

The outer membrane of the mitochondrion is shown in a glowing pinkish-purple hue, with a smooth and shiny surface. The inner membrane is rendered in a contrasting blue color and is highly convoluted, forming folds known as cristae, which increase the surface area for chemical reactions.

Within the inner membrane, the matrix space appears to be a darker blue, and there are small yellow and red speckles scattered throughout, possibly representing molecules or atoms. The entire image has a soft glow to it and is set against a dark background that helps to accentuate the mitochondrion's luminescence.

The artistic approach to this render gives the mitochondrion a more abstract and visually engaging look compared to the typical representations found in textbooks. The image does not show the detailed molecular structures or the exact internal components, but rather focuses on the overall shape and form of the organelle with an emphasis on color and lighting.

OpenAI vs. Local

- have been using OpenAI API to use LLM's
 - costs money
 - less selection

OpenAI vs. Local

- have been using OpenAI API to use LLM's
 - costs money
 - less selection
- Local Open-Source LLM
 - Mistral
 - Gemini
 - Llama



LM Studio



Download LM Studio for M1/M2/M3

0.2.20



Download LM Studio for Windows

0.2.20



Download LM Studio for Linux (Beta)

0.2.20

Download



LM Studio

Load Model

Start Local
Server

Create
Agents and
Run Task

Large Language Models

BCV

Application Layer



Infrastructure Layer



What is Function Calling?

- so far, we let **agents** produce answers and execute code
- sometimes we have functions we want to be executed which are user made
- maybe we have **specific currency exchange**, or want to ensure we always **save the last result to a file**

Just a Python Function

python

 Copy code

```
def reverse_string(input_string):  
    return input_string[::-1]  
  
# Example usage:  
original_string = "hello"  
reversed_string = reverse_string(original_string)  
print("Original:", original_string)  
print("Reversed:", reversed_string)
```

- **define this function in code**
- **and attach to an agent**

AutoGen Function Calling

```
useragent.register_for_execution
```

```
coder.register_for_llm()
```

```
currency_converter(...)
```

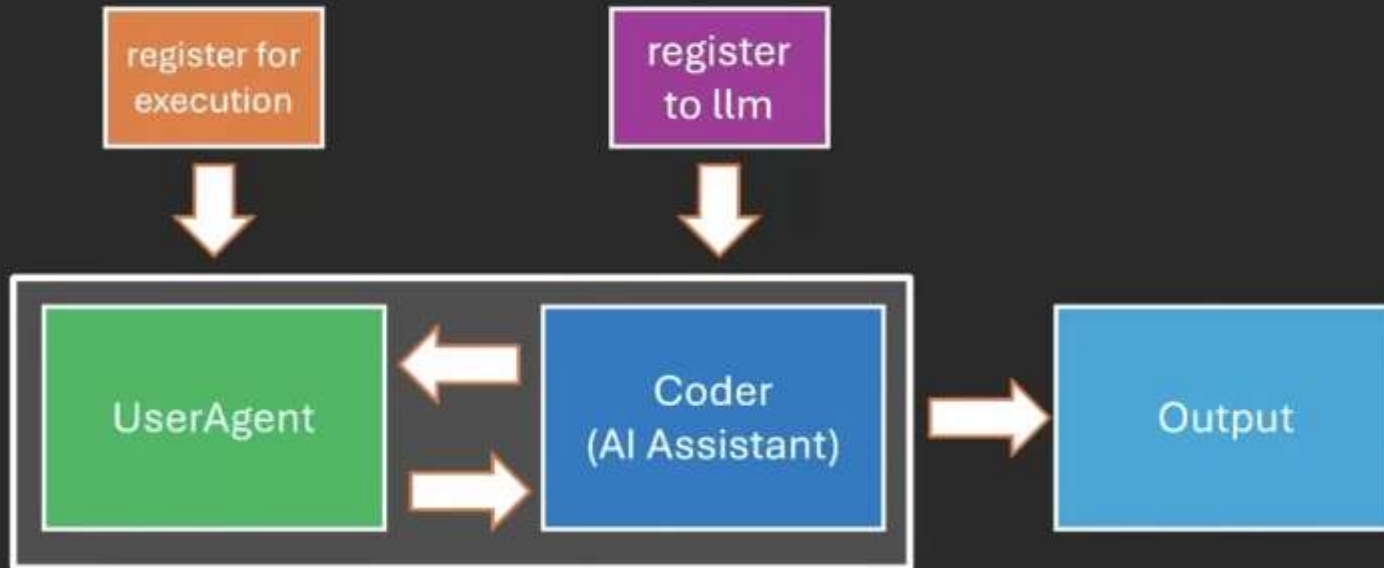
register for
execution

register
to llm

UserAgent

Coder
(AI Assistant)

Output



What are **TOOLS**?

UserAgent



get_time
get_weather

Assistant



“what time is it in NYC?”

UserAgent



Assistant



“what time is it in NYC?”

YES

Use a
tool?

NO

Choose the
correct tool

get_weather
get_time

Execute the
tool

Response

LLM

“The current time in New
York City is 15:33 on
April 24, 2024”

